

CSCI203 – Algorithms and data structures

Recursion

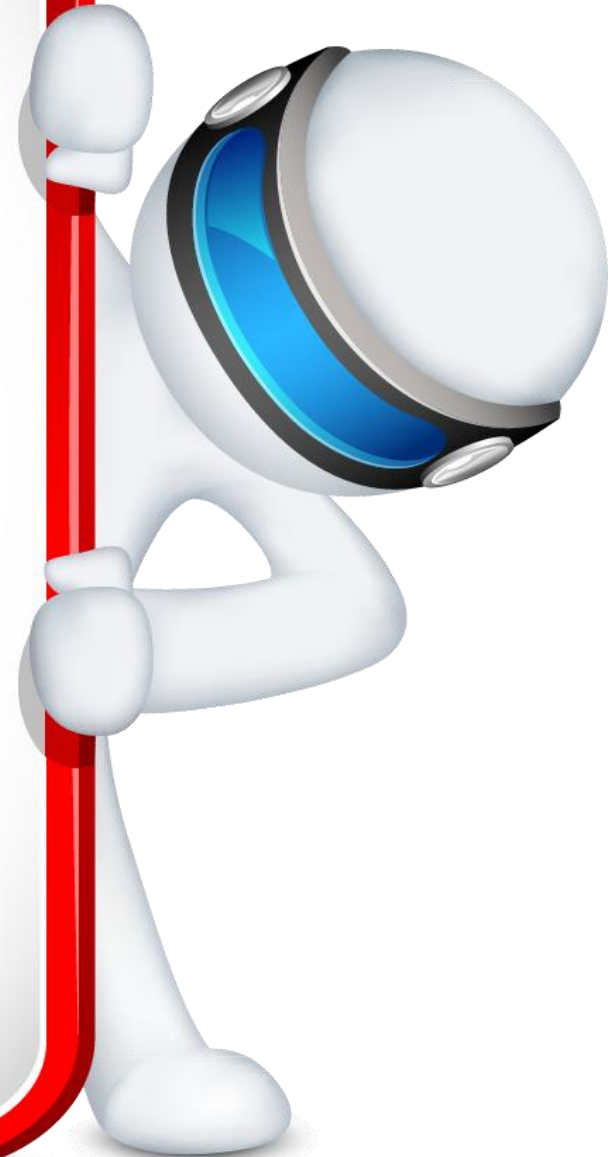
Sionggo Japit

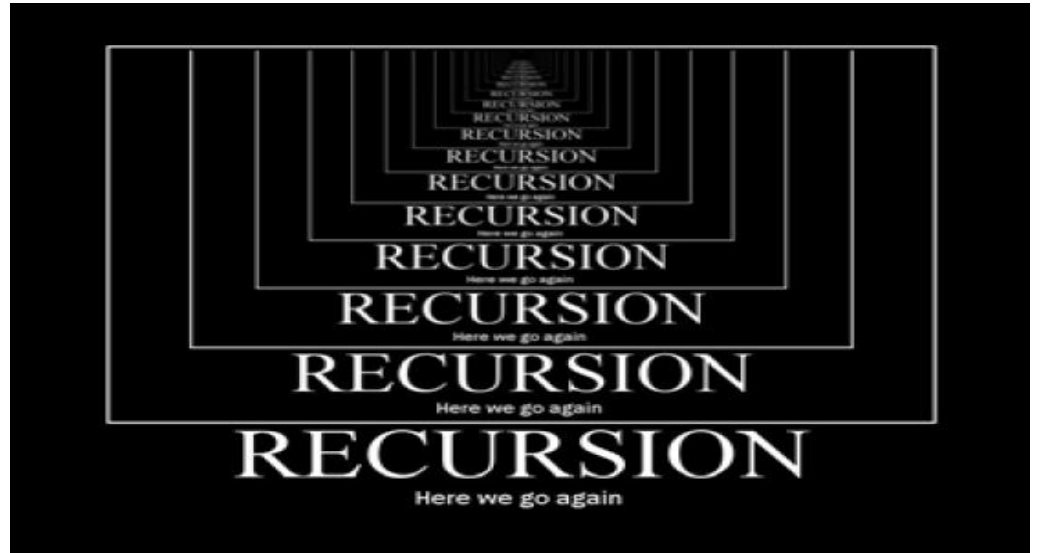
sjapit@uow.edu.au

31 December 2025

Lecture Outline

- Recursion
- Recurrence Relation

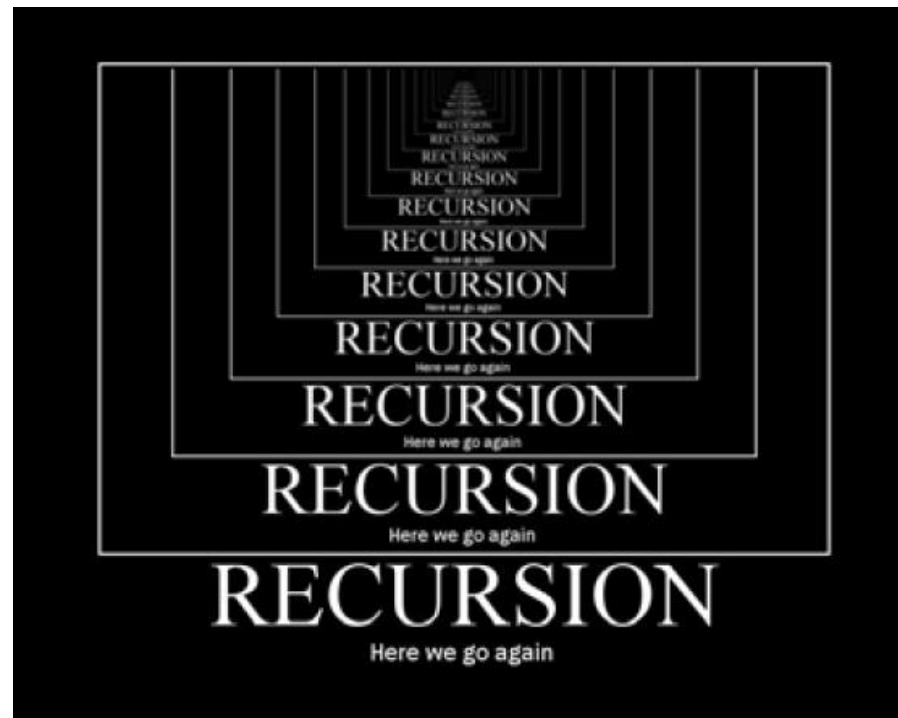




Recursion

Recursion

- Recursion is a repetitive process in which an algorithm calls itself.



Recursion

- Each recursive call solves an **identical**, but **smaller, problem**.
- A test for the **base case** enables the recursive calls to stop.
- Eventually, one of the smaller problems must be the base case.
 - Begin by testing for a set of base cases (there should be at least one).
 - Every possible chain of recursive calls must eventually reach a base case, and the **handling of each base case should not** use recursion.

Recursion

Fibonacci numbers

- 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, ...
- Definition:

$$f(n) = \begin{cases} 0, & n = 0 \\ 1, & n = 1 \\ f_{n-1} + f_{n-2}, & n \geq 2 \end{cases}$$

The Fibonacci numbers are the sequence of numbers $\{F_n\}_{n=1}^{\infty}$ defined by the linear recurrence equation

$$F_n = F_{n-1} + F_{n-2}$$

With $F_0 = 0$ and $F_1 = 1$.

Recursion

- It appears that a simple recursive algorithm will do a good job of calculating f_n .

```
algorithm fib (val num <integer>)  
  if (num is 0 or num is 1)  
    return num  
  end if  
  return (fib (num -1) +  
         fib (num -2))  
end fib
```

```
long fib (long num)  
{  
  if (num == 0 || num == 1)  
    return num;  
  return (fib (num -1) +  
          fib (num -2));  
}
```

Recursion

- How good is this solution?
 - Note that we calculate the same function more than once.
 - This can't be a good thing.
 - Just how bad is it?



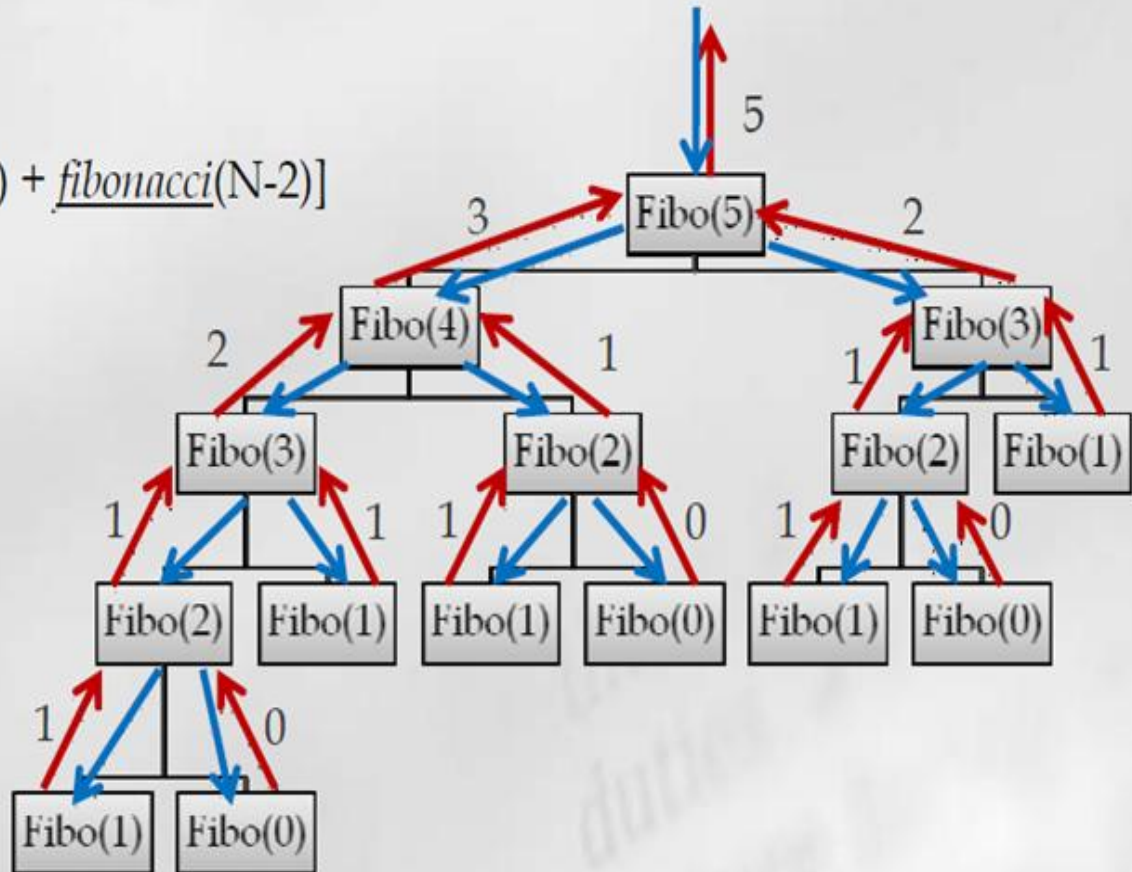
Recursion

- Consider fibrec(5)
 - To compute fibrec(5), the algorithm needs fibrec(4) and fibrec(3)
 - But fibrec(4) needs fibrec(3) and fibrec(2)
 - And fibrec(3) needs fibrec(2) and fibrec(1)
 - Finally fibrec(2) needs fibrec(1) and fibrec(0)
 - So, to evaluate fibrec(5) we evaluate fibrec(4) once, fibrec(3) twice, fibrec(2) three times, fibrec(1) five times and fibrec(0) three times

Recursion

```
function fibonacci(N)
```

1. if (N<2) then
2. return N
3. return [fibonacci(N-1) + fibonacci(N-2)]



Recursive Tree

Recursion

- The following table tabulates the number of invocations (calls) that a recursively implemented Fibonacci function invokes itself.

No.	Calls	No.	Calls	No.	Calls
1	1	8	67	15	1973
2	3	9	109	20	21,891
3	5	10	177	25	242,785
4	9	11	287	30	2,692,573
5	15	12	465	35	29,860,703
6	25	13	753	40	331,160,281
7	41	14	1219		

Recursion

- The time taken to calculate f_n with the recursive algorithm is proportional to f_n .
- It can be proved that the runtime of the recursive algorithm is $O((3/2)^n)$. [Note: the proof is out of the scope of this module. However, if you are interested, you can refer to [https://en.wikipedia.org/wiki/Fibonacci_number#Relation to the golden ratio](https://en.wikipedia.org/wiki/Fibonacci_number#Relation_to_the_golden_ratio) for the proof.]
- The runtime grows exponentially, the recursive algorithm is obviously not practical!

Recursion

- Fibonacci numbers - An iterative Algorithm

```
function fibiter(n)
  i = 1; j = 0
  for k = 1 to n do
    j = i + j
    i = j - i
  return j
```

- Is this solution better? What is the running time complexity?
 - The time taken to evaluate f_n is linearly proportionate to n ; that is, the running time is $O(n)$.

Recursion

- Comparing the algorithms (results are from empirical studies.)

N	10	20	30	50	60
fibrec	7.14 msec	3.26 msec	2.98 msec	53.22 msec	158 min
fibiter	9.78 msec	2.68 msec	7.99 msec	4.03 msec	12.96 msec

- In fact, there is an algorithm that evaluates f_n in time proportional to $\lg(n)$.

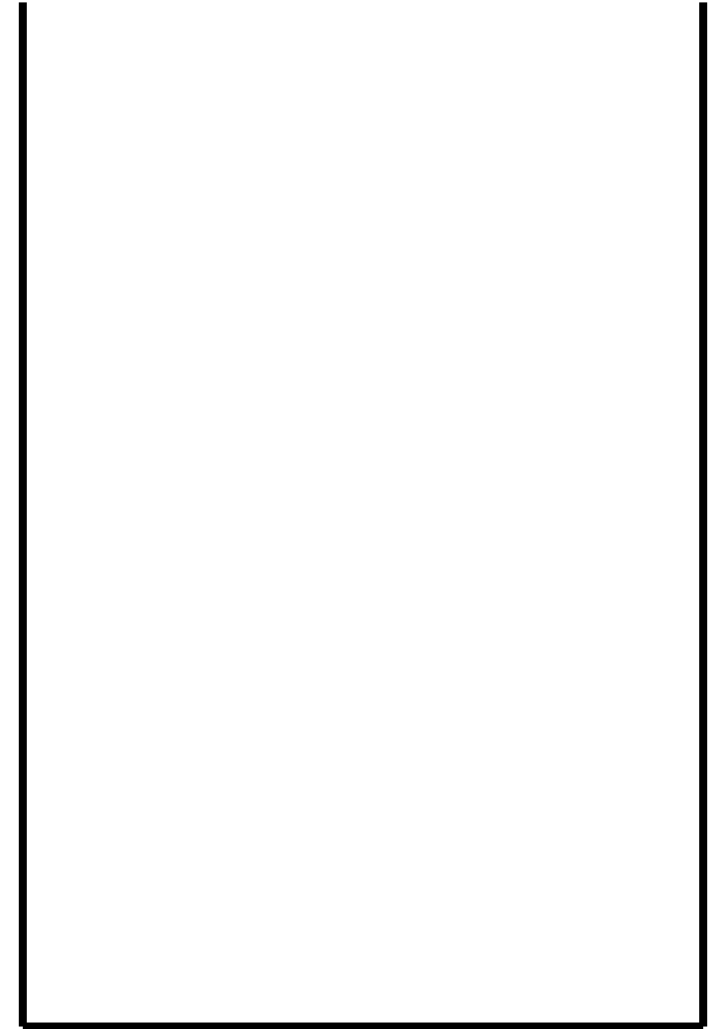
Recursion and Stacks

- Every recursive call has an Activation Record (AR) that includes the spaces needed for parameters, local variables, a return value, and a place to return control when done. This AR is created and placed on the system stack.
- When a copy of the recursively called function is completed, its AR instance (ARI) is popped / removed from the system stack.

Recursion and Stacks

Algorithm: Fact(n)

```
{  
  if n = 0  
    return 1;  
  else  
    return n * Fact( n - 1 )  
}
```



System Stack

Recursion and Stacks

```
Algorithm: Fact(n)
{
  if n = 0
    return 1;
  else
    return n * Fact( n - 1 )
}
```

A

n=3,
Fact(3-1)

3	:n
A	:Return address

System Stack

Recursion and Stacks

Algorithm: Fact(n)

```
{  
  if n = 0  
    return 1;  
  else  
    return n * Fact( n - 1 )  
}
```

B

A

n=2,
Fact(2-1)

n=3,
Fact(3-1)

2 :n

B :Return address

3 :n

A :Return address

System Stack

Recursion and Stacks

Algorithm: Fact(n)

{

if n = 0

return 1;

else

return n * Fact(n - 1)

}

C
B
A

n=1,
Fact(1-1)

n=2,
Fact(2-1)

n=3,
Fact(3-1)

1

:n

C

:Return address

2

:n

B

:Return address

3

:n

A

:Return address

System Stack

Recursion and Stacks

Algorithm: Fact(n)

```
{  
  if n = 0  
    return 1;  
  else  
    return n * Fact( n - 1 )  
}
```

D
C
B
A

n=0,
Fact(0-1)

n=1,
Fact(1-1)

n=2,
Fact(2-1)

n=3,
Fact(3-1)

0	:n
D	:Return address

1	:n
C	:Return address

2	:n
B	:Return address

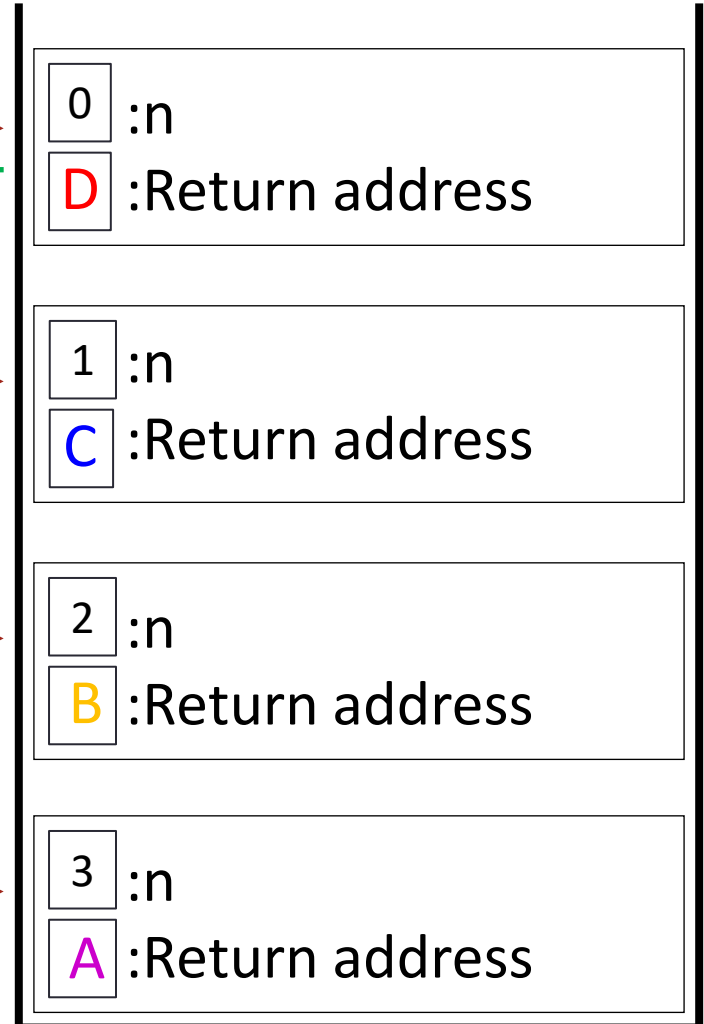
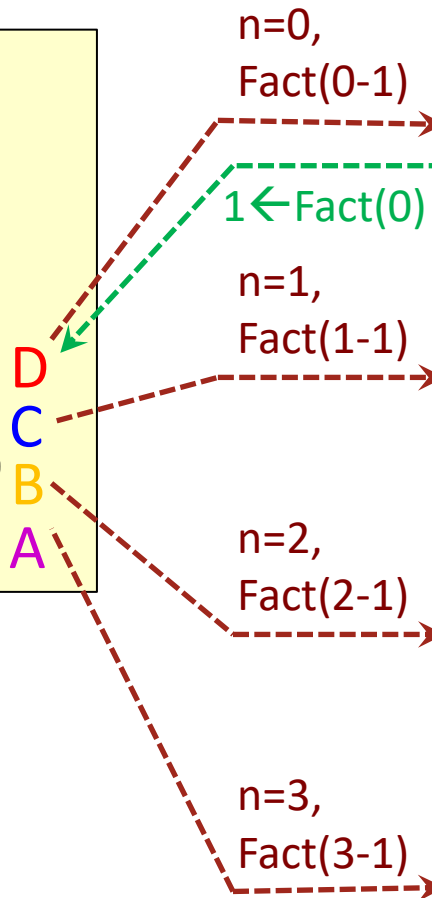
3	:n
A	:Return address

System Stack

Recursion and Stacks

Algorithm: Fact(n)

```
{  
  if n = 0  
    return 1;  
  else  
    return n * Fact( n - 1 )  
}
```

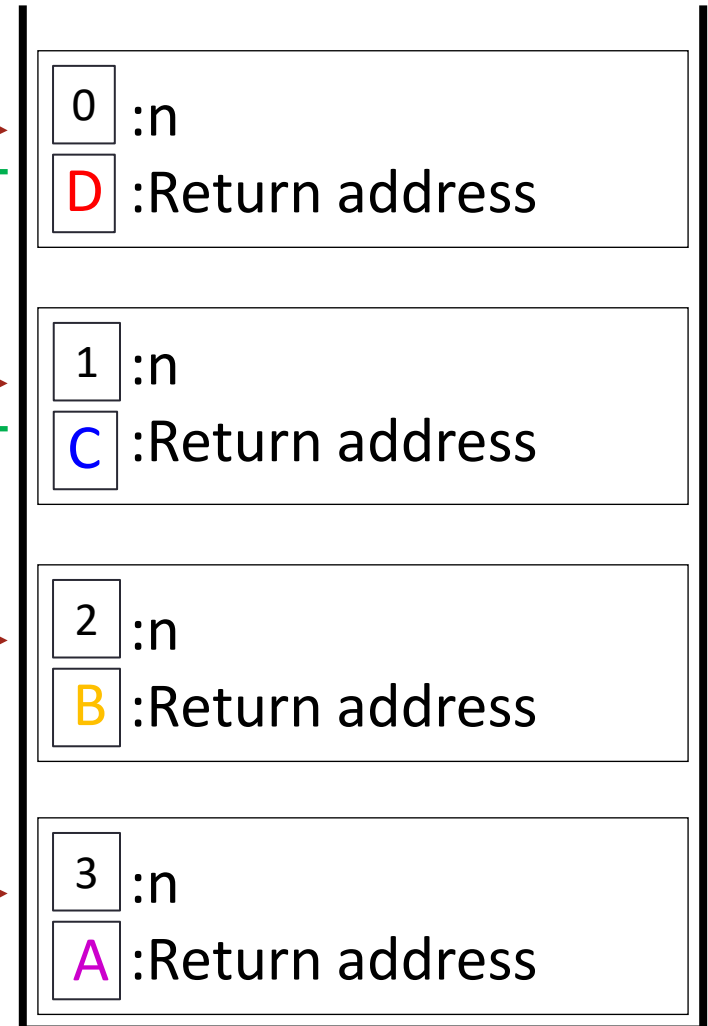
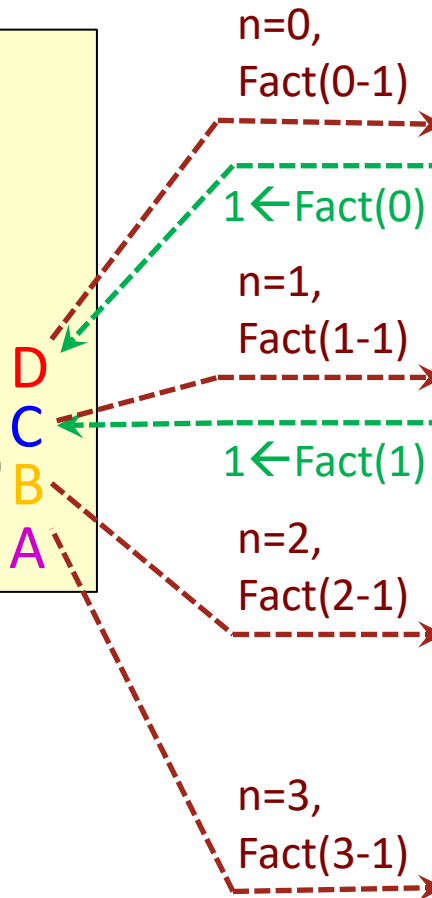


System Stack

Recursion and Stacks

Algorithm: Fact(n)

```
{  
  if n = 0  
    return 1;  
  else  
    return n * Fact( n - 1 )  
}
```

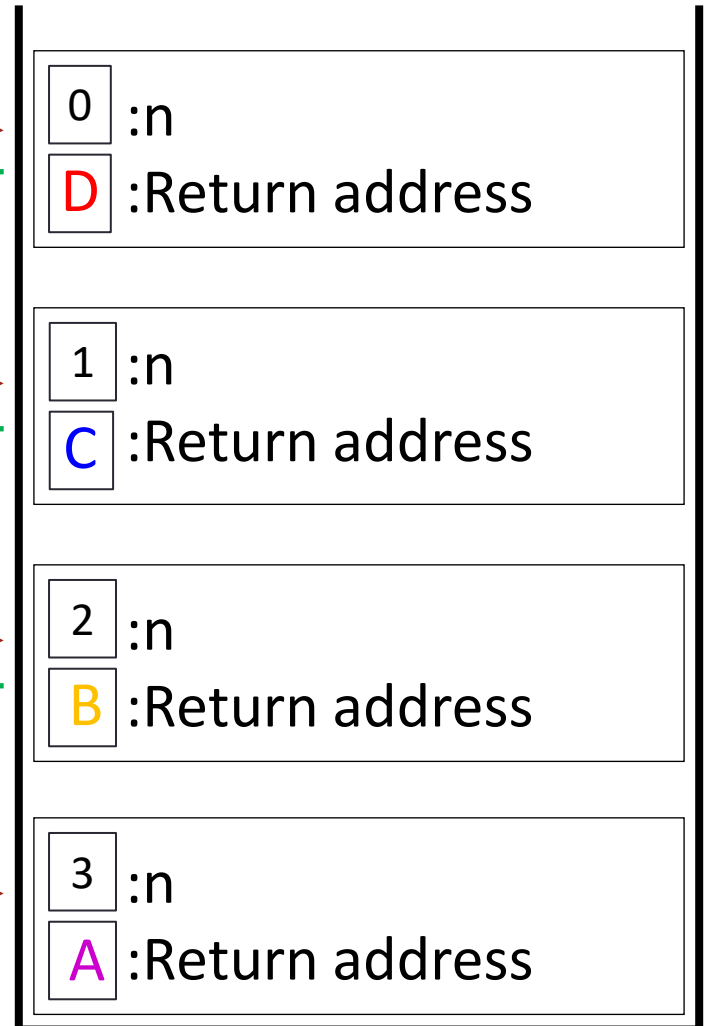
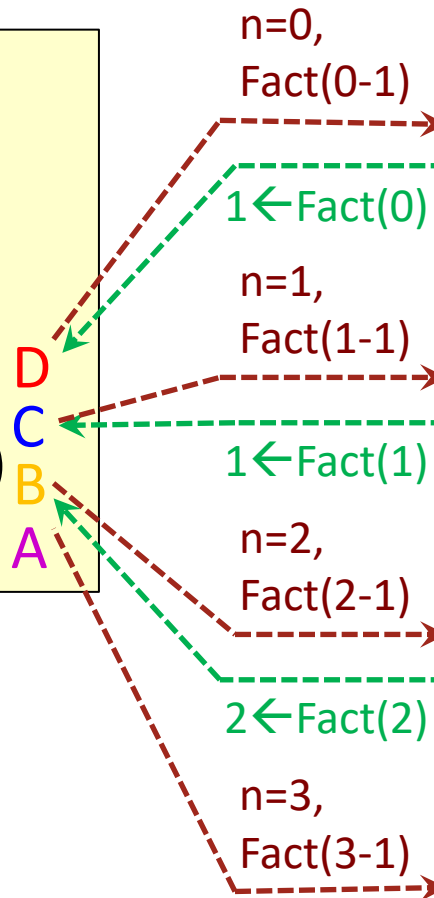


System Stack

Recursion and Stacks

Algorithm: Fact(n)

```
{  
  if n = 0  
    return 1;  
  else  
    return n * Fact( n - 1 )  
}
```

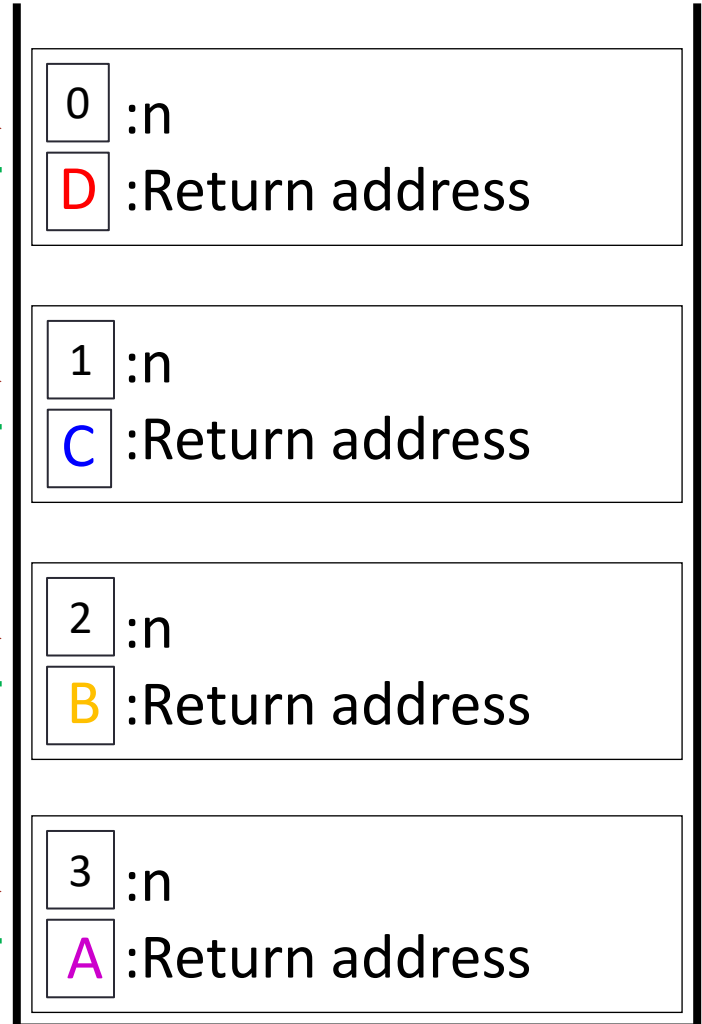
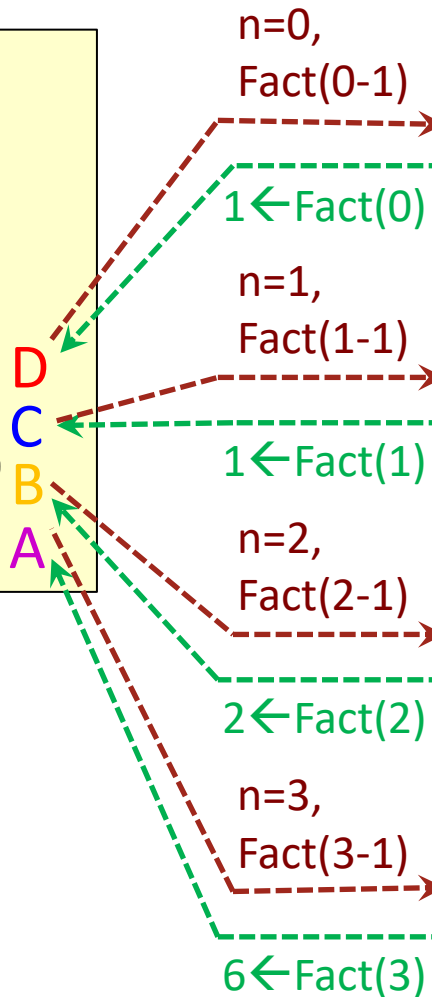


System Stack

Recursion and Stacks

Algorithm: Fact(n)

```
{  
  if n = 0  
    return 1;  
  else  
    return n * Fact( n - 1 )  
}
```



System Stack

Type of Recursion

- Recursion algorithms can be implemented in various forms:
 - Linear recursion
 - Linear, Tail recursion,
 - Linear, Non-tail recursion
 - Non-linear recursion
 - Non-linear, non-tail recursion.

Type of Recursion

- Linear vs Non-linear

A recursion function is **linear recursion** if each execution of it “**calls itself at most once**,” that is, each activation of it involves at most one new activation.

Otherwise, it is a **non-linear recursion**.

Type of Recursion - examples

- Implementation of Fibonacci function using linear recursion

```
int FibNumL (int n, int x, int y)
{
    // Base conditions
    if (n == 1)
        return y;
    else
        // Recursive call of function linearly
        return ( FibNumL (n - 1, y, x+y) );
}
```

Type of Recursion - examples

- Implementation of Fibonacci function using linear recursion

```
int FibNumL (int n, int x, int y)
{
    // Base conditions
    if (n == 1)
        return y;
    else
        // Recursive call of function linearly
        return ( FibNumL (n - 1, y, x+y) );
}
```

At one time there is only one recursive function is called, hence linear.

Type of Recursion - examples

- Implementation of Fibonacci function using non-linear recursion

```
int FibNumNL (int n)
{
    // Base conditions
    if (n < 1)
        return -1;
    if (n == 1 || n == 2)
        return 1;

    // Recursive call by binary method
    return ( FibNumNL (n - 1) + FibNumNL (n - 2) );
}
```

Type of Recursion - examples

- Implementation of Fibonacci function using non-linear recursion

```
int FibNumNL (int n)
{
    // Base conditions
    if (n < 1)
        return -1;
    if (n == 1 || n == 2)
        return 1;

    // Recursive call by binary method
    return ( FibNumNL (n - 1) + FibNumNL (n - 2) );
}
```

At one time more than one recursive functions are called, hence non-linear.

Type of Recursion

- Tail vs Non-tail

A recursion function is **tail recursion** if each execution of it *either*:

- defines the function outright (base cases) or
- calls itself “only” with different values for arguments (or parameters).

and the recursive call is the last statement in a method.

Otherwise, it is a **non-tail recursion**.

Type of Recursion - examples

- Implementation of Factorial function using tail recursion

```
int FactorialT (int n, long accumulator)
{
    // Base conditions
    if (n == 0)
        return accumulator;
    else
        // Recursive call linearly
        return ( FactorialT (n - 1, n * accumulator) );
}
```


Type of Recursion - examples

- Implementation of Factorial function using tail recursion

```
int FactorialT (int n, long accumulator)
```

```
{  
    // Base conditions  
    if (n == 0)  
        return accumulator;  
    else  
        // Recursive call linearly  
        return ( FactorialT (n - 1, n * accumulator) );  
}
```

The function calls itself with different values for arguments, and the recursive call is the last statement in the function; hence it is a tail recursion.

Type of Recursion - examples

- Implementation of Factorial function using non-tail recursion

```
int FactorialNT (long n)
{
    // Base conditions
    if (n < 0)
        return -1;
    if (n == 0)
        return 1;
    else
        // Recursive call linearly
        return ( n * FactorialNT (n - 1) );
}
```

Type of Recursion - examples

- Implementation of Factorial function using non-tail recursion

```
int FactorialNT (long n)
{
    // Base conditions
    if (n < 0)
        return -1;
    if (n == 0)
        return 1;
    else
        // Recursive call linearly
        return ( n * FactorialNT (n - 1) );
}
```

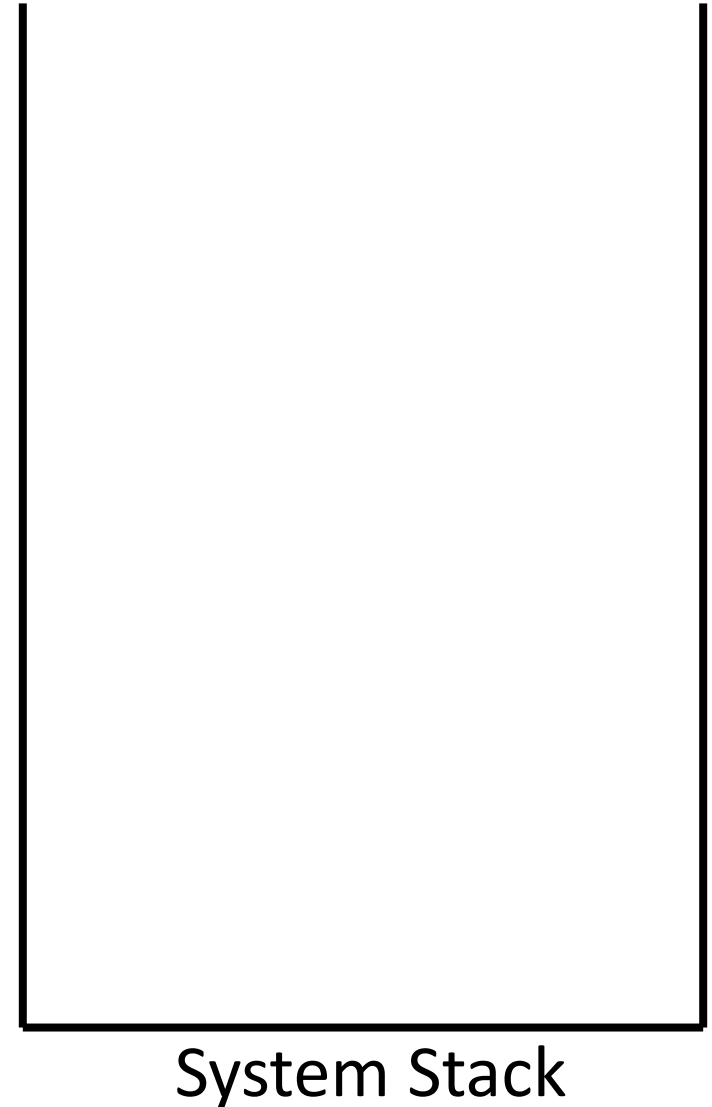
The function calls itself with different values for arguments, however, there is still multiplication operation is needed; hence it is a non-tail recursion.

Recursion and Stacks

Activation records of
tail-recursion function

$X = \text{Fact}(3,1);$

```
Algorithm: Fact(int n, long a)
{
    if n == 0
        return a;
    else
        return Fact( n - 1, n * a )
}
```



Recursion and Stacks

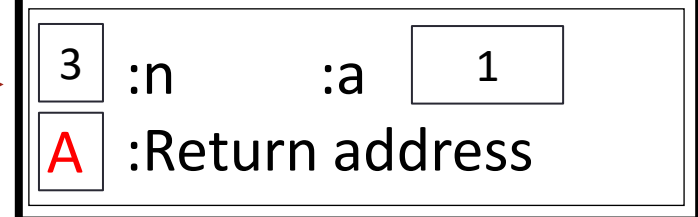
Activation records of
tail-recursion function

$X = \text{Fact}(3, 1);$

```
Algorithm: Fact(int n, long a)
{
    if n == 0
        return a;
    else
        return Fact( n - 1, n * a )
}
```

$n=3, a=1$
 $\text{Fact}(3-1, 3 * 1)$

A ----->



System Stack

Recursion and Stacks

Activation records of
tail-recursion function

$X = \text{Fact}(3,1);$

```
Algorithm: Fact(int n, long a)
{
  if n == 0
    return a;
  else
    return Fact( n - 1, n * a )
}
```

$n=2, a=3$

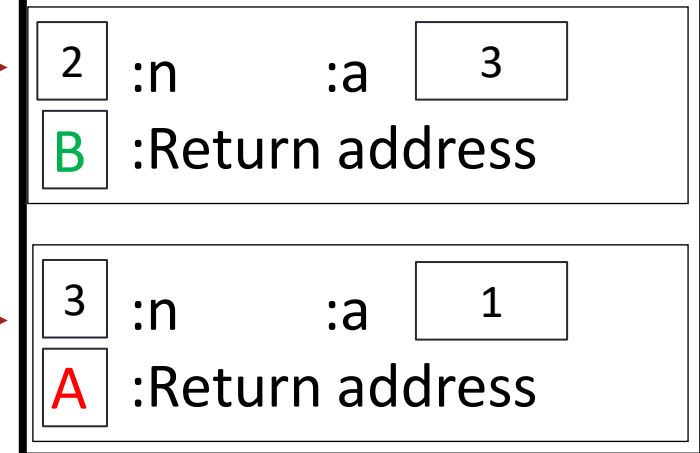
$\text{Fact}(2-1, 2 * 3)$

B

$n=3, a=1$

$\text{Fact}(3-1, 3 * 1)$

A



System Stack

Recursion and Stacks

Activation records of
tail-recursion function

$X = \text{Fact}(3, 1);$

```
Algorithm: Fact(int n, long a)
{
    if n == 0
        return a;
    else
        return Fact( n - 1, n * a )
}
```

$n=1, a=6$

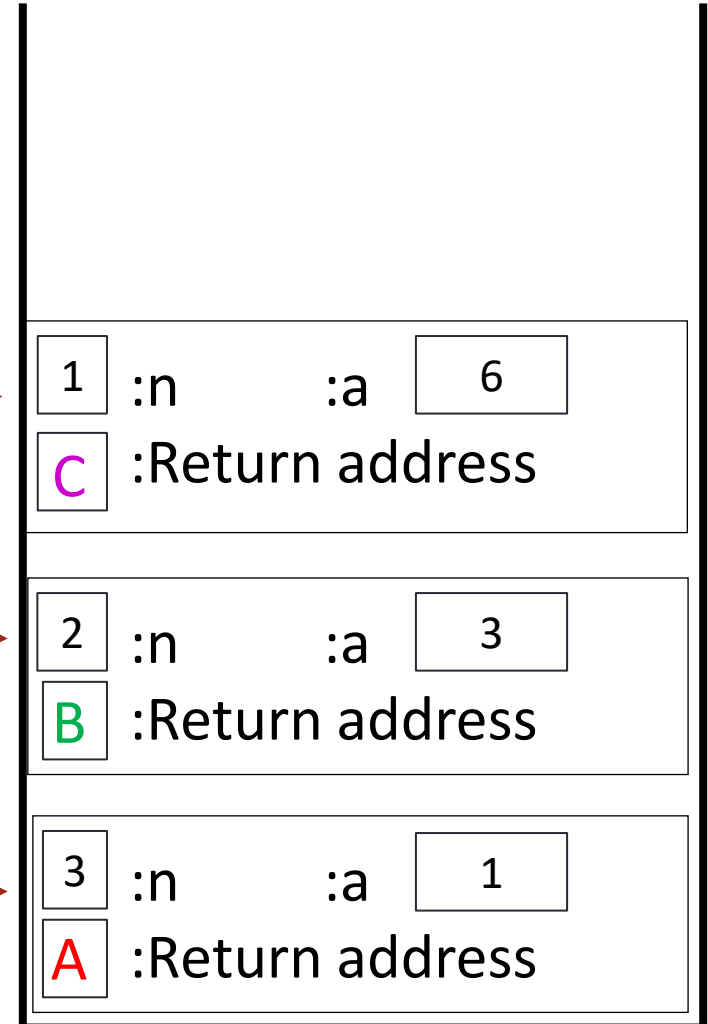
Fact(1-1, 1 * 6)

$n=2, a=3$

Fact(2-1, 2 * 3)

$n=3, a=1$

Fact(3-1, 3 * 1)



System Stack

Recursion and Stacks

Activation records of
tail-recursion function

$X = \text{Fact}(3, 1);$

```
Algorithm: Fact(int n, long a)
{
    if n == 0
        return a;
    else
        return Fact( n - 1, n * a )
}
```

$n=0, a=6$

Fact(1-1, 1 * 6)

$n=1, a=6$

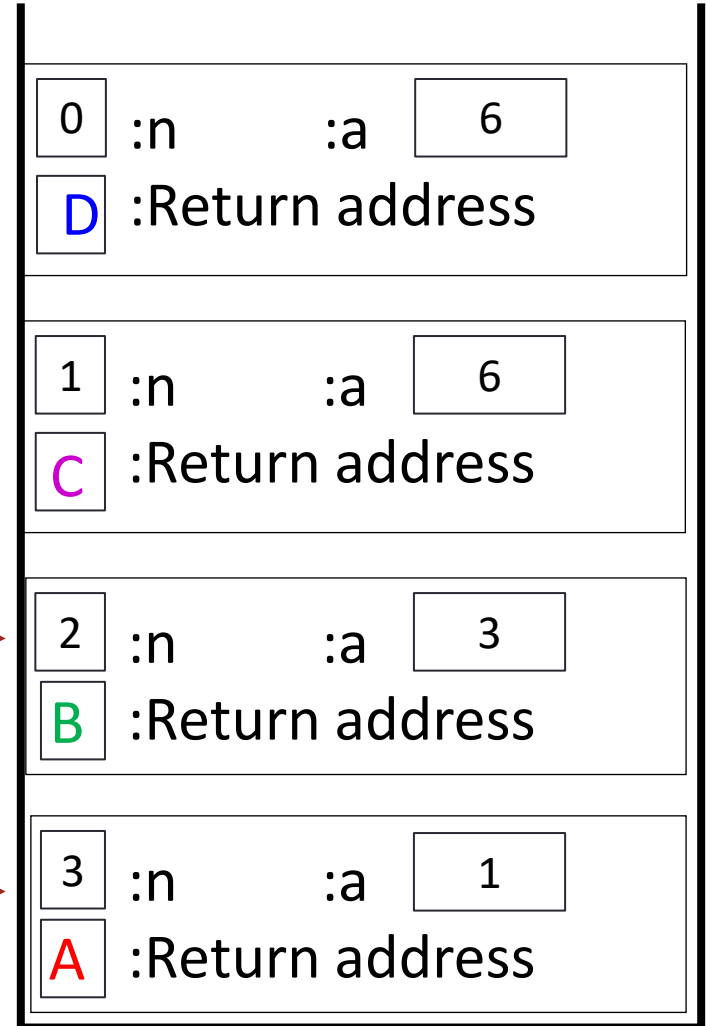
Fact(1-1, 1 * 6)

$n=2, a=3$

Fact(2-1, 2 * 3)

$n=3, a=1$

Fact(3-1, 3 * 1)



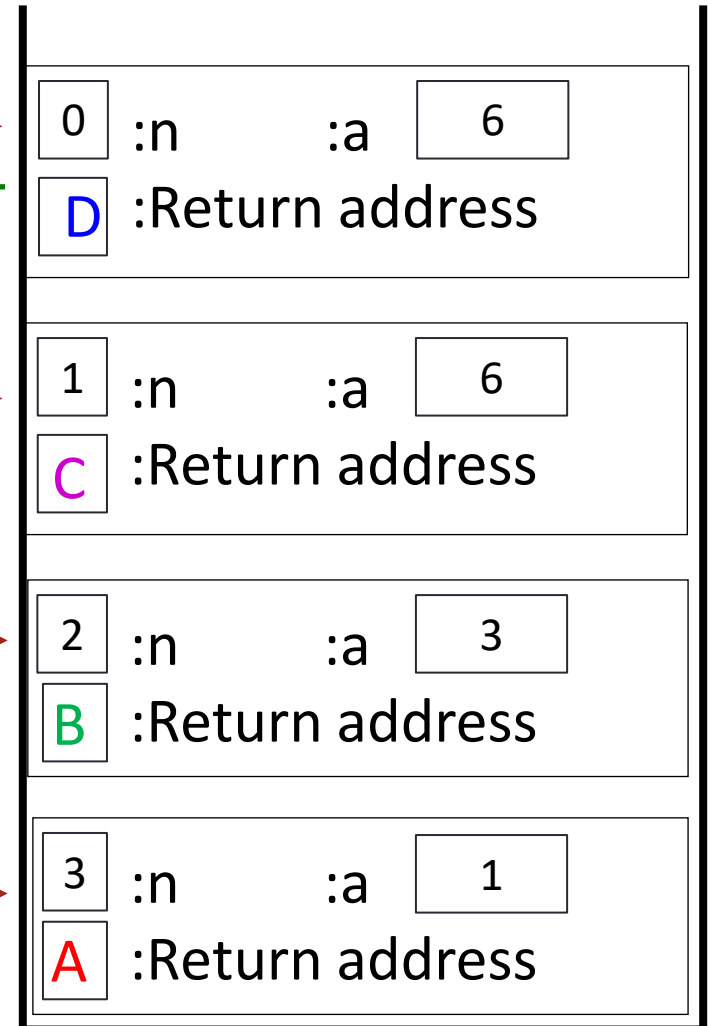
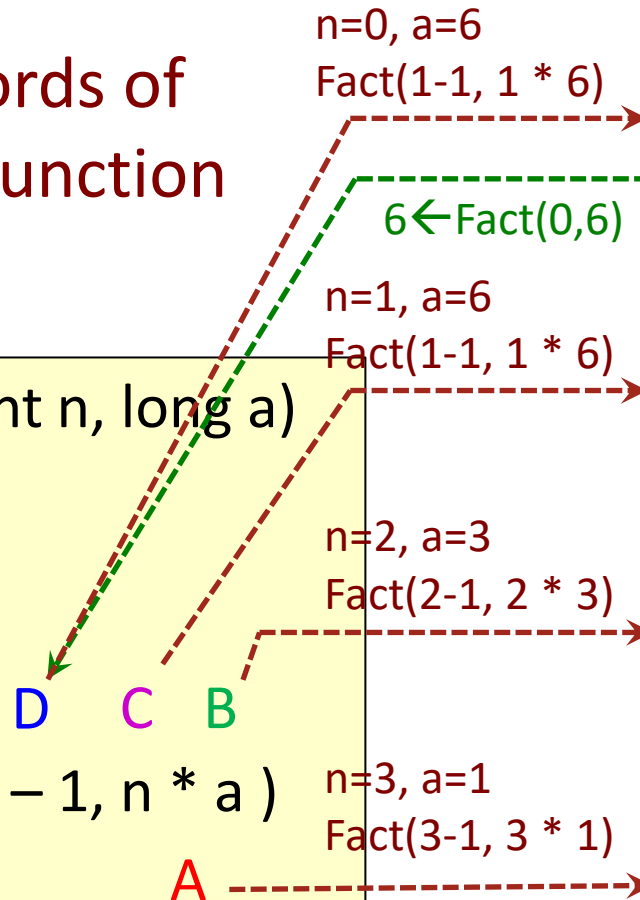
System Stack

Recursion and Stacks

Activation records of
tail-recursion function

$X = \text{Fact}(3, 1);$

```
Algorithm: Fact(int n, long a)
{
    if n == 0
        return a;
    else
        return Fact( n - 1, n * a )
}
```



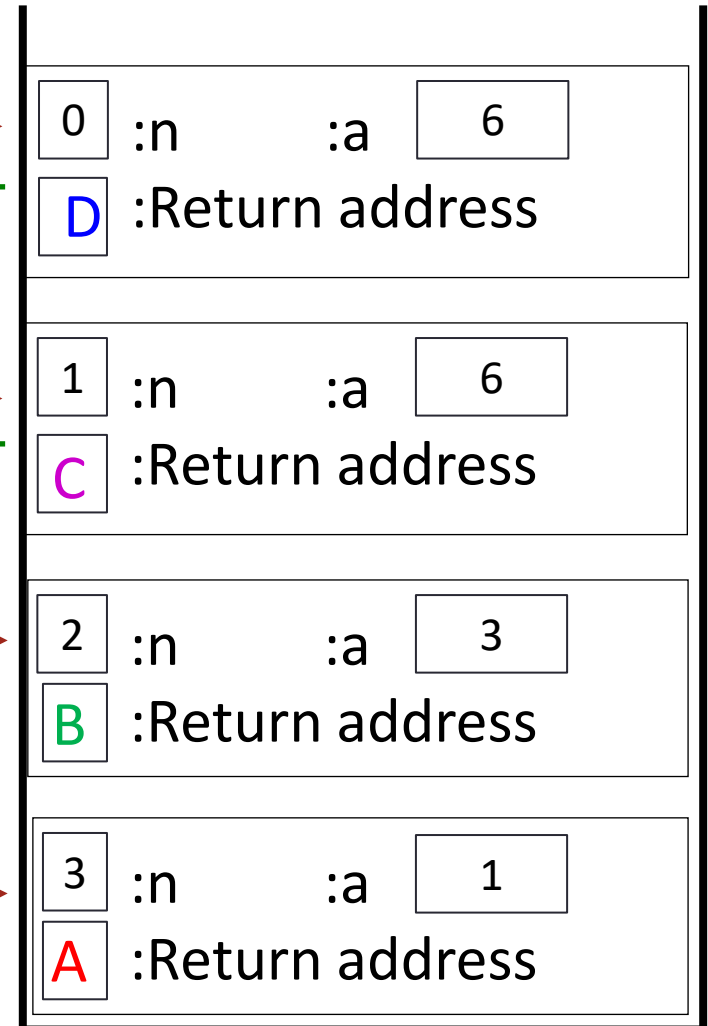
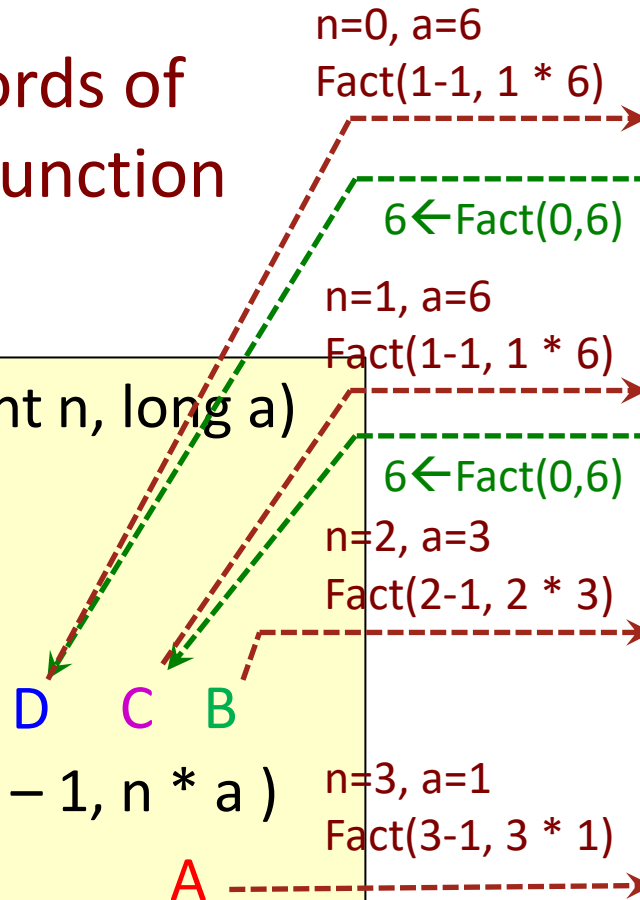
System Stack

Recursion and Stacks

Activation records of
tail-recursion function

$X = \text{Fact}(3, 1);$

```
Algorithm: Fact(int n, long a)
{
  if n == 0
    return a;
  else
    return Fact( n - 1, n * a )
}
```



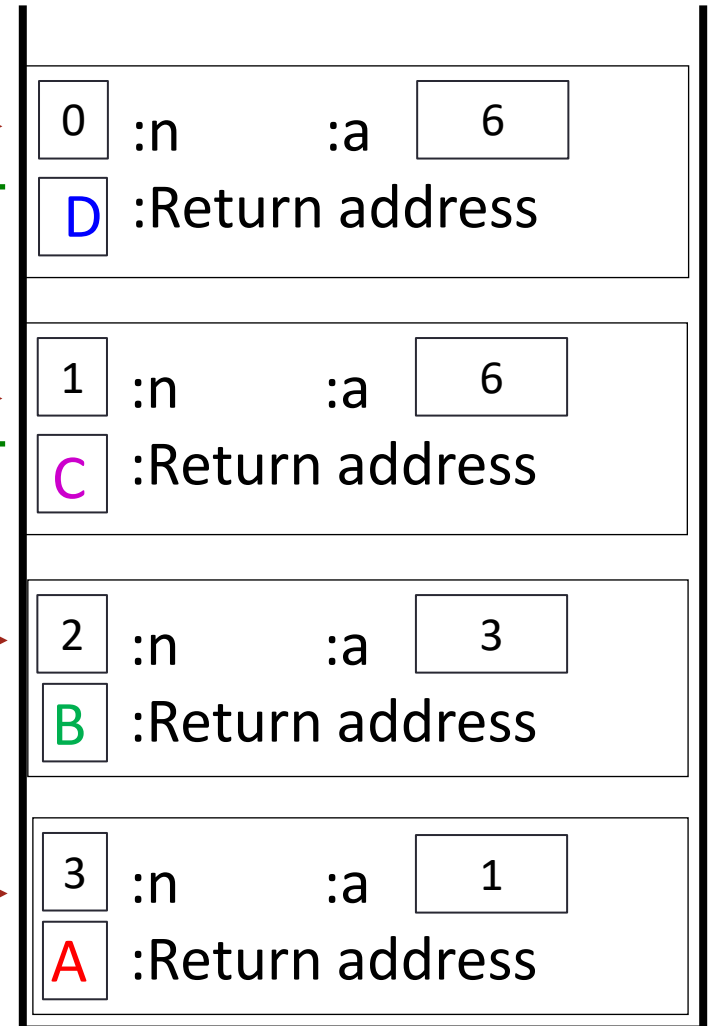
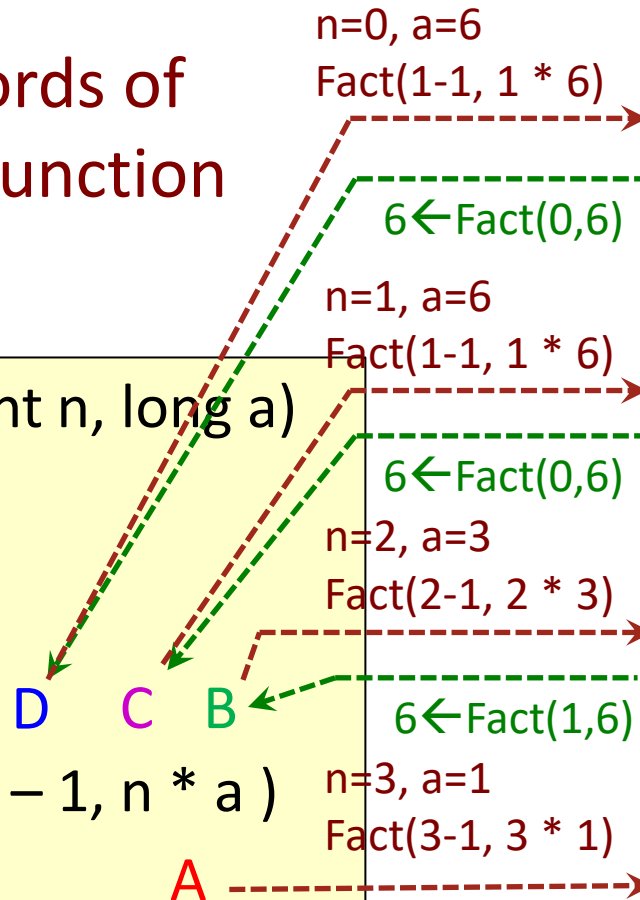
System Stack

Recursion and Stacks

Activation records of
tail-recursion function

$X = \text{Fact}(3, 1);$

```
Algorithm: Fact(int n, long a)
{
  if n == 0
    return a;
  else
    return Fact( n - 1, n * a )
}
```



System Stack

Recursion and Stacks

Activation records of tail-recursion function

```
X = Fact (3,1);
```

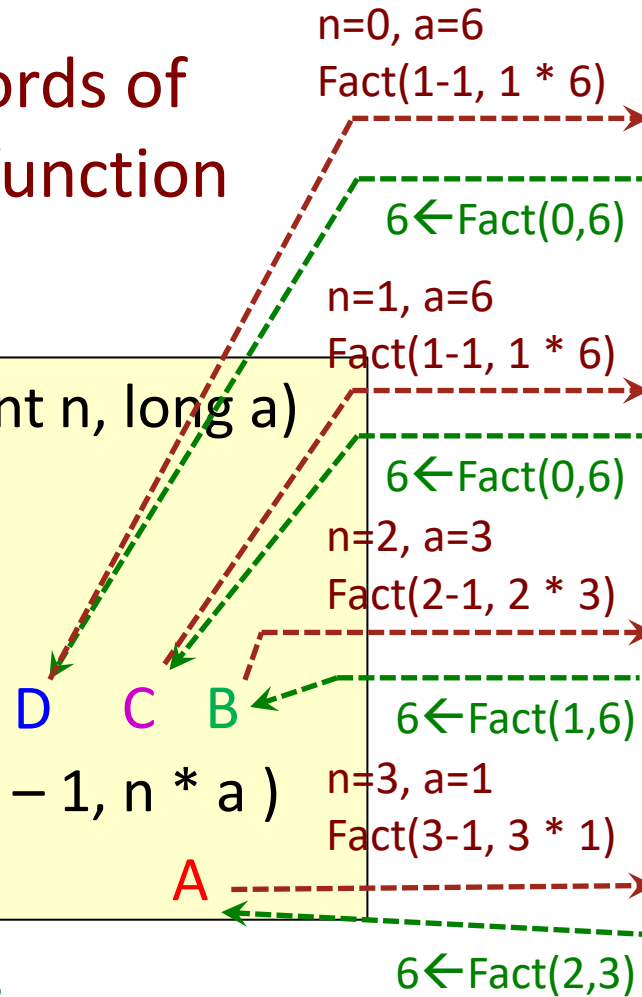
```

Algorithm: Fact(int n, long a)
{
    if n == 0
        return a;
    else
        return Fact( n - 1, n * a )
}

```

Diagram illustrating the recursive calls for `Fact(4, 1)`. The call stack is represented by nodes A, B, C, and D. Node A is the initial call, which calls B. B calls C, which calls D. D returns 1 to C, which returns 2 to B, which returns 24 to A. The return values are shown in red dashed boxes.

```
6 ← Fact (3,1);
```



The diagram illustrates a stack with four frames, each containing a return address and a pointer to the next frame. The stack grows downwards from higher memory addresses at the top to lower memory addresses at the bottom.

Frame Index	Return Address	Next Frame Address
0	D	6
1	C	6
2	B	3
3	A	1

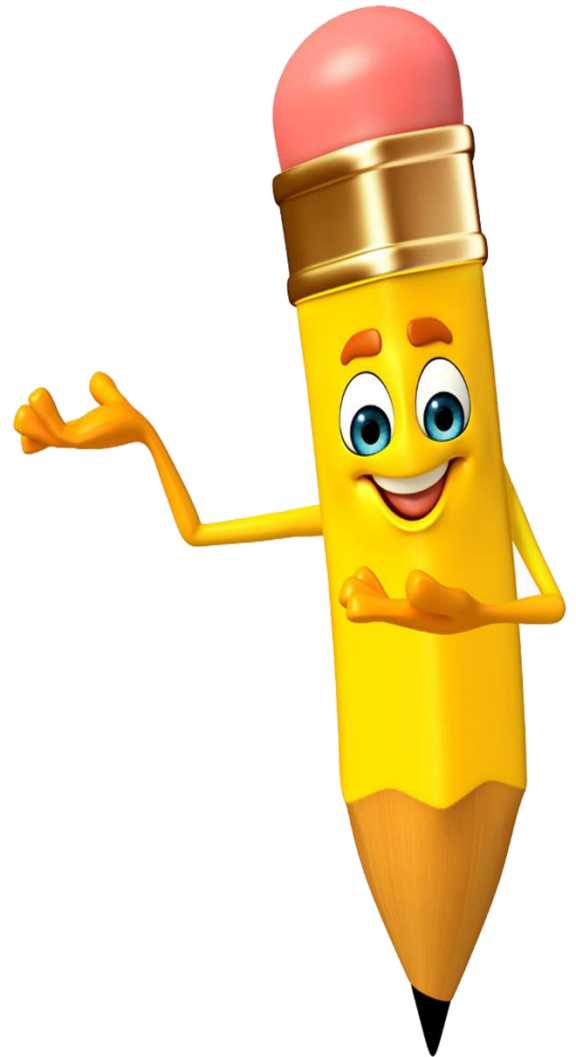
System Stack

Recurrence Relation

$$T(0) = 2$$

$$T(1) = 2$$

$$T(n) = 2T(n - 1) + 1$$



What is a recurrence relation?

- A recurrence relation, denoted as $T(n)$, is a recursive form of equation that involves some constant and integer variable n .
- Like all recursive functions, it has both recursive case and base case.
- For example:

$$f(n) = \begin{cases} 0, & x = 0 \\ 1, & x = 1 \\ f_{n-1} + f_{n-2}, & x \geq 2 \end{cases}$$

$$T(0) = 2$$

$$T(1) = 2$$

$$T(n) = 2T(n - 1) + 1$$

What is a recurrence relation?

- A recurrence relation, denoted as $T(n)$, is a recursive form of equation that involves some constant and integer variable n .
- Like all recursive functions, it has both recursive case and base case.
- For example:

Recurrence relations

$$f(n) = \begin{cases} 0, & x = 0 \\ 1, & x = 1 \\ f_{n-1} + f_{n-2}, & x \geq 2 \end{cases}$$

$$T(0) = 2$$

$$T(1) = 2$$

$$T(n) = 2T(n - 1) + 1$$

What is recurrence relation?

$$T(0) = 2$$

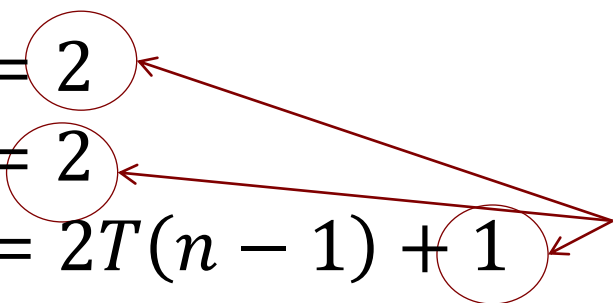
$$T(1) = 2$$

$$T(n) = 2T(n - 1) + 1$$

What is recurrence relation?

$$T(0) = 2$$

$$T(1) = 2$$

$$T(n) = 2T(n - 1) + 1$$


The term(s) of the relation that **does not** contain T is called the **base case** of the recurrence relation.

What is recurrence relation?

$$T(0) = 2$$

$$T(1) = 2$$

$$T(n) = 2T(n - 1) + 1$$

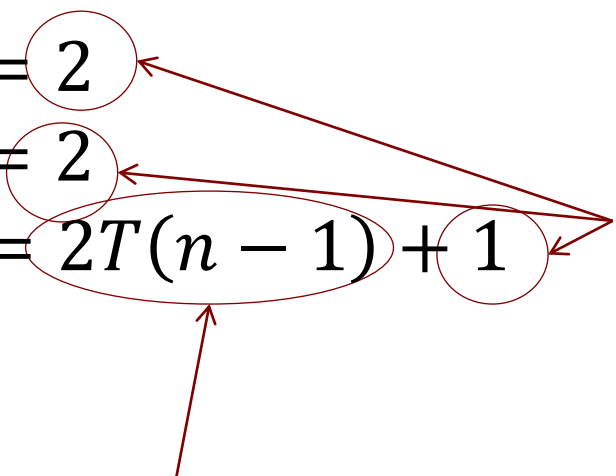
The term(s) of the relation that **does not** contain T is called the **base case** of the recurrence relation.

The term(s) of the relation that contains T is called the **recurrent or recursive case**.

What is recurrence relation?

$$T(0) = 2$$

$$T(1) = 2$$

$$T(n) = 2T(n-1) + 1$$


The term(s) of the relation that **does not** contain T is called the **base case** of the recurrence relation.

The term(s) of the relation that contains T is called the **recurrent or recursive case**.

- Recurrence relations are useful for expressing the running times, that is, the number of basic operations executed, of recursive algorithms.

Forming recurrence relations

- For a given recursive method, the base case and the recursive case of its recurrence relation correspond directly to the base case and the recursive case of the method.
- For example, what is the recurrence relation for the following recursive function?

```
public void f (int n) {  
    if (n == 0)  
        return 1;  
    else  
        return n * f (n - 1);  
}
```

Forming recurrence relations

- For a given recursive method, the base case and the recursive case of its recurrence relation correspond directly to the base case and the recursive case of the method.
- For example, what is the recurrence relation for the following recursive function?

```
public void f (int n) {  
    if (n == 0)  
        return 1;  
    else  
        return n * f (n - 1);  
}
```

What function is this? 😊

Forming recurrence relations

```
public void f (int n) {  
    if (n = 0)  
        return 1;  
    else  
        return n * f (n - 1);  
}
```

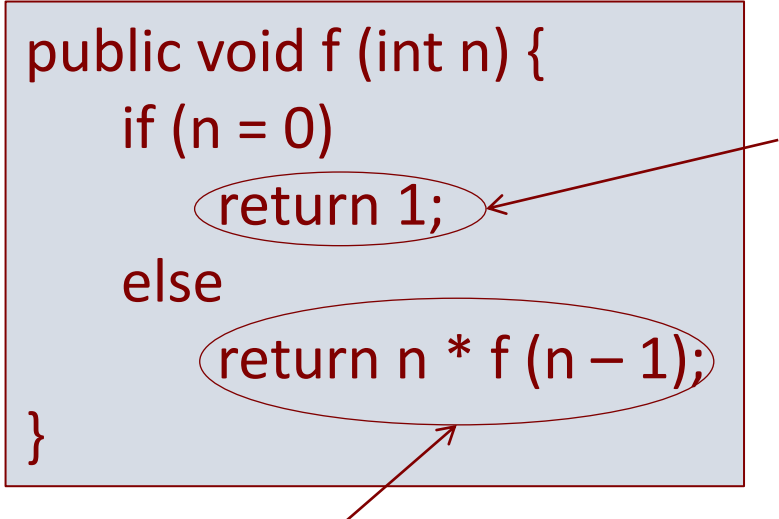
Forming recurrence relations

```
public void f (int n) {  
    if (n = 0)  
        return 1;  
    else  
        return n * f (n - 1);  
}
```

The base case is reached when $n = 0$. The function performs **ONE** comparison and a return statement. Hence, the number of operation is some constant, that is $T(0) = a$.

Forming recurrence relations

```
public void f (int n) {  
    if (n = 0)  
        return 1;  
    else  
        return n * f (n - 1);  
}
```



The base case is reached when $n = 0$. The function performs **ONE** comparison and a return statement. Hence, the number of operation is some constant, that is $T(0) = a$.

When $n > 0$, the function performs **ONE** recursive call with a parameter $n - 1$, then a multiplication and a return statement. Hence, the number of operations consists of a recursive term plus some constant, that is $T(n) = T(n - 1) + b$.

Forming recurrence relations

```
public void f (int n) {  
    if (n = 0)  
        return 1;  
    else  
        return n * f (n - 1);  
}
```

The base case is reached when $n = 0$. The function performs **ONE** comparison and a return statement. Hence, the number of operation is some constant, that is $T(0) = a$.

When $n > 0$, the function performs **ONE** recursive call with a parameter $n-1$, then a multiplication and a return statement. Hence, the number of operations consists of a recursive term plus some constant, that is $T(n) = T(n-1) + b$.

Hence, the recurrence relations are:

$T(0) = a$ for some constant a

$T(n) = T(n-1) + b$ for a constant b and a recursive term

Forming recurrence relations

- Another example:

```
public int myFunction (int n) {  
    if (n == 1)  
        return 1;  
    else  
        return 2 * myFunction (n / 2) + myFunction (n / 2) + 1;  
}
```

- The base case is reached when $n == 1$. The function performs one comparison and **ONE** return statement. Hence, $T(1)$ is some constant c .
- When $n > 1$, the function performs **TWO** recursive calls, each with the parameter $n/2$, and some constant number of basic operations.

Forming recurrence relations

Hence, the recurrence relations are:

$$T(1) = c \quad \text{for some constant } c$$

$$T(n) = 2T\left(\frac{n}{2}\right) + b \quad \text{for a constant } b \text{ and a recursive term}$$

Solving recurrence relations

- There are four methods to solve recurrence relations that represent the running time of recursive methods:
 - Recursion tree method
 - **Iteration method (unrolling and summing)**
 - **Master method**
 - Substitution method
- We will discuss the iteration method and the master method here.

Solving recurrence relations – Iteration method

Steps:

- Expand the recurrence relation
- Express the expansion as a summation by plugging the recurrence back into itself until we see a pattern.
- Evaluate the summation.

Solving recurrence relations – Iteration method

Example:

Form and solve the recurrence relation for the running time of factorial function and hence determine its big-O complexity:

```
public void factorial (int n) {  
    if (n == 0)  
        return 1;  
    else  
        return n * factorial (n - 1);  
}
```

Solving recurrence relations – Iteration method

```
public void factorial (int n) {  
    if (n == 0)  
        return 1;  
    else  
        return n * factorial (n - 1);  
}
```

Solving recurrence relations – Iteration method

```
public void factorial (int n) {  
    if (n == 0)  
        return 1;  
    else  
        return n * factorial (n - 1);  
}
```

$$T(0) = c$$

$$T(n) = b + T(n-1)$$

$$= b + b + T(n-2)$$

$$= b + b + b + T(n-3)$$

...

$$= kb + T(n-k)$$

Solving recurrence relations – Iteration method

```
public void factorial (int n) {  
    if (n == 0)  
        return 1;  
    else  
        return n * factorial (n - 1);  
}
```

$$T(0) = c$$

$$T(n) = b + T(n-1)$$

$$= b + b + T(n-2)$$

$$= b + b + b + T(n-3)$$

...

$$= kb + T(n-k)$$

When $k = n$, we have:

$$T(n) = nb + T(n-n)$$

$$= bn + T(0)$$

$$T(n) = bn + c \text{ since } T(0) = c$$

Solving recurrence relations – Iteration method

What does $T(n) = bn + c$ indicate?

Solving recurrence relations – Iteration method

What does $T(n) = bn + c$ indicate?

The running time complexity of the factorial function is $O(n)$.

Analysis of recursion binary search

- Search for x in a **sorted** array A .

Binary – Search(A, p, q, x)

if $p \geq q$ return -1 ;

$r = \lfloor (p + q) / 2 \rfloor$

if $x = A[r]$ return r

else if $x < A[r]$ Binary – Search($A, p, r - 1, x$)

else Binary – Search($A, r + 1, q, x$)

- The initial call is Binary-Search($A, 1, n, x$)

Recursion

Write down the **recurrence equation** which describes the running time of the algorithm Binary-Search($A, 1, n, x$) as a function of n .

```
Binary – Search( $A, p, q, x$ )
  if  $p \geq q$  return  $-1$ ;
   $r = \lfloor (p + q) / 2 \rfloor$ 
  if  $x = A[r]$  return  $r$ 
  else if  $x < A[r]$  Binary – Search( $A, p, r - 1, x$ )
  else Binary – Search( $A, r + 1, q, x$ )
```

Recursion

Write down the **recurrence equation** which describes the running time of the algorithm Binary-Search($A, 1, n, x$) as a function of n .

Binary – Search(A, p, q, x)

if $p \geq q$ return -1 ;

$r = \lfloor (p + q) / 2 \rfloor$

if $x = A[r]$ return r

else if $x < A[r]$ Binary – Search($A, p, r - 1, x$)

else Binary – Search($A, r + 1, q, x$)

$$T(n) = c \quad \text{if } n = 1$$

$$T(n) = k + T\left(\left\lfloor \frac{n}{2} \right\rfloor\right) \quad \text{if } n > 0$$

Recursion

What is the upper-bound (big-O) complexity of the algorithm $\text{Binary-Search}(A, 1, n, x)$?

$$\begin{aligned}T(n) &= b + T\left(\frac{n}{2}\right) \\&= b + b + T\left(\frac{n}{(2)^2}\right) \\&= b + b + b + T\left(\frac{n}{(2)^3}\right) \\&\dots \\&= kb + T\left(\frac{n}{(2)^k}\right)\end{aligned}$$

Recursion

when $\frac{n}{2^k} = 1$, we have $n = 2^k$, and this happens when $p = q$.

Since $n = 2^k$, we have $k = \lg n$ (Note: $\lg n = \log_2 n$)

Substituting k with $\lg n$ into $T(n) = kb + T\left(\frac{n}{(2)^k}\right)$,
we have $T(n) = (\lg n)b + T\left(\frac{n}{(2)^{\lg n}}\right)$.

Recursion

Hence the total cost $T(n) = b(\lg n) + T\left(\frac{n}{2^{\lg n}}\right)$

$= b(\lg n) + T(1)$ (Note: $2^{\lg n} = n$)

$= b(\lg n) + c$

$\Rightarrow O(\lg n)$

Master Theorem

Test methods



Determine the run-time complexity – Master Theorem

- The Master method is a **general method to determine the run-time complexity of algorithm** that arise frequently in divide and conquer strategies, which have the following recurrence relation form:

$$T(n) = aT\left(\frac{n}{b}\right) + \Theta(n^c \log_b^p n)$$

Where:

- a is a constant and ≥ 1
- b is a constant and > 1
- c is a constant and ≥ 0
- *and p is some real number.*

Determine the run-time complexity – Master Theorem

- The constant a is the number of sub-problems into which a problem of size n was divided, and for each division, it is divided into b .
- For example, a list of size n is divided into 5 sub-lists, and each time a list or sub-list is divided into *two*. Hence $a = 5$ and $b = 2$.

Determine the run-time complexity – Master Theorem

- Recurrence relation of the form $T(n) = aT\left(\frac{n}{b}\right) + \Theta(n^c \log_b^p n)$ can be bounded asymptotically as follows:

Case 1: if $\frac{a}{b^c} > 1$, then $T(n) = \Theta(n^{\log_b a})$

Determine the run-time complexity – Master Theorem

Case 2: if $\frac{a}{b^c} = 1$, and

i. if $p < -1$, then $T(n) = \Theta(n^{\log_b a})$

ii. if $p = -1$, then $T(n) = \Theta(n^{\log_b a} \times \log_b \log_b n)$

iii. if $p > -1$, then $T(n) = \Theta(n^{\log_b a} \times \log_b^{p+1} n)$

Determine the run-time complexity – Master Theorem

Case 3: if $\frac{a}{b^c} < 1$, and

i. if $p < 0$, then $T(n) = O(n^c)$

ii. if $p \geq 0$, then $T(n) = \Theta(n^c \times \log_b^p n)$

Determine the run-time complexity – Master Theorem

The master theorem cannot be applied if:

- $T(n)$ is not **monotone**, e.g., $T(n) = \sin n$.
- $f(n)$ is not **polynomial**, e.g., $T(n) = 2T(n/2) + 2^n$.
- b cannot be expressed as a constant (no change), e.g., $T(n) = T(\sqrt{n})$.
- Another important note: the Master Theorem **does not** solve a recurrence relation.

Determine the run-time complexity – Master Theorem

- We will use the following examples to understand how the master theorem is used to determine the run-time complexity of the various forms of recurrence relations.

Determine the run-time complexity of the following recurrence equation using Master Theorem (if possible):

- $T(n) = 3T(n/2) + n^2$
- $T(n) = 4T\left(\frac{n}{2}\right) + n^2$
- $T(n) = 16T\left(\frac{n}{4}\right) + n$
- $T(n) = 2T\left(\frac{n}{2}\right) + n \lg n$
- $T(n) = 0.5T\left(\frac{n}{2}\right) + \frac{1}{n}$
- $T(n) = 64T\left(\frac{n}{8}\right) - n^2 \lg n$

$$T(n) = 3T\left(\frac{n}{2}\right) + n^2$$

$$a = 3, b = 2, c = 2, p = 0$$

$$\frac{a}{b^c} = \frac{3}{2^2} < 1, \text{ and } p = 0, \text{ then}$$

$$\begin{aligned} T(n) &= \Theta(n^c \log_b^p n) \\ &= \Theta(n^2 \log_2^0 n) \\ &= \Theta(n^2) \end{aligned}$$

$$T(n) = 4T\left(\frac{n}{2}\right) + n^2$$

$$a = 4, b = 2, c = 2, p = 0$$

$$\frac{a}{b^c} = \frac{4}{2^2} = 1, \text{ and } p > -1, \text{ then}$$

$$T(n) = \Theta\left(n^{\log_b a} \log_b^{p+1} n\right)$$

$$= \Theta\left(n^{\log_2 4} \log_2^1 n\right)$$

$$= \Theta(n^2 \log_2 n)$$

$$T(n) = 16T\left(\frac{n}{4}\right) + n$$

$$a = 16, b = 4, c = 1, p = 0$$

$$\frac{a}{b^c} = \frac{16}{4^1} > 1, \text{ then}$$

$$T(n) = \Theta(n^{\log_b a})$$

$$= \Theta(n^{\log_2 4})$$

$$= \Theta(n^2)$$

$$T(n) = 2T\left(\frac{n}{2}\right) + n \log_2 n$$

$$a = 2, b = 2, c = 1, p = 1$$

$$\frac{a}{b^c} = \frac{2}{2^1} = 1, \text{ and } p > -1 \text{ then}$$

$$\begin{aligned} T(n) &= \Theta(n^{\log_b a} \log_b^{p+1} n) \\ &= \Theta(n^{\log_2 2} \log_2^2 n) \\ &= \Theta(n \log_2^2 n) \end{aligned}$$

$$T(n) = 0.5T\left(\frac{n}{2}\right) + \frac{1}{n}$$

- $a = 0.5, b = 2, c = -1, f(n) = \frac{1}{n}$
- a is less than 1, hence master theorem cannot be applied to determine the running time complexity of this recurrence relation. Expansion and substitution will be used instead.

$$T(n) = 0.5T\left(\frac{n}{2}\right) + \frac{1}{n}$$

- $a = 0.5, b = 2, c = -1, f(n) = \frac{1}{n}$
- a is less than 1, hence master theorem cannot be applied to determine the running time complexity of this recurrence relation. Expansion and substitution will be used instead.

$$T(n) = \frac{1}{2} T\left(\frac{n}{2}\right) + \frac{1}{n}$$

$$T(n) = \frac{1}{2} \left[\frac{1}{2} T\left(\frac{n}{2^2}\right) + \frac{1}{\frac{n}{2}} \right] + \frac{1}{n} \quad \text{Continue to next slide...}$$

$$T(n) = 0.5T\left(\frac{n}{2}\right) + \frac{1}{n}$$

Continue from previous slide...

$$T(n) = \frac{1}{2} \left[\frac{1}{2} T\left(\frac{n}{2^2}\right) + \frac{1}{\frac{n}{2}} \right] + \frac{1}{n}$$

$$T(n) = \frac{1}{2^2} T\left(\frac{n}{2^2}\right) + \left(\frac{1}{2}\right) \frac{2}{n} + \frac{1}{n} \quad \text{Continue to next slide...}$$

$$T(n) = 0.5T\left(\frac{n}{2}\right) + \frac{1}{n}$$

Continue from previous slide...

$$T(n) = \frac{1}{2^2} T\left(\frac{n}{2^2}\right) + \left(\frac{1}{2}\right) \frac{2}{n} + \frac{1}{n}$$

$$T(n) = \frac{1}{2^2} \left[\frac{1}{2} T\left(\frac{n}{2^3}\right) + \frac{1}{\frac{n}{2}} \right] + \left(\frac{1}{2}\right) \frac{2}{n} + \frac{1}{n}$$

$$T(n) = \frac{1}{2^3} T\left(\frac{n}{2^3}\right) + \left(\frac{1}{2^2}\right) \frac{2}{n} + \left(\frac{1}{2}\right) \frac{2}{n} + \frac{1}{n}$$

The recurrence relation can be generalized as

$$T(n) = \frac{1}{2^k} T\left(\frac{n}{2^k}\right) + \left[\frac{1}{2^k} + \frac{1}{2^{k-1}} + \frac{1}{2^{k-2}} + \cdots + \frac{1}{2^2} + \frac{1}{2^1} \right] \left(\frac{2}{n}\right) + \frac{1}{n} \quad \cdots eq(1)$$

The recursive call will continue and stop when $\left(\frac{n}{2^k}\right) = 1$. Solving the equality, we have $n = 2^k$ and hence $k = \lg n$.

Substituting k into $eq(1)$, we have

$$T(n) = \frac{1}{2^{\lg n}} T\left(\frac{n}{2^{\lg n}}\right) + \left[\frac{1}{2^{\lg n}} + \frac{1}{2^{(\lg n)-1}} + \cdots + \frac{1}{2} \right] \left(\frac{2}{n}\right) + \frac{1}{n}$$

Continue to next slide...

Continue from previous slide...

$$T(n) = \frac{1}{2^{\lg n}} T\left(\frac{n}{2^{\lg n}}\right) + \left[\frac{1}{2^{\lg n}} + \frac{1}{2^{(\lg n)-1}} + \cdots + \frac{1}{2} \right] \left(\frac{2}{n}\right) + \frac{1}{n}$$

$$T(n) = \frac{1}{n} T\left(\frac{n}{n}\right) + \left[\frac{1}{2^{\lg n}} + \frac{1}{2^{(\lg n)-1}} + \cdots + \frac{1}{2} \right] \left(\frac{2}{n}\right) + \frac{1}{n}$$

$$T(n) = \frac{1}{n} T(1) + \left[\frac{1}{2^{\lg n}} + \frac{1}{2^{(\lg n)-1}} + \cdots + \frac{1}{2} \right] \left(\frac{2}{n}\right) + \frac{1}{n}$$

Continue from previous slide...

$$T(n) = \frac{1}{n}T(1) + \left[\frac{1}{2^{\lg n}} + \frac{1}{2^{(\lg n) - 1}} + \cdots + \frac{1}{2} \right] \left(\frac{2}{n} \right) + \frac{1}{n}$$

$\Theta\left(\frac{1}{2^{\lg n}}\right) = \Theta\left(\frac{1}{n}\right)$. Does not seem to be sensible algorithm complexity because as n increases, the complexity reduces (becomes less complex).

$$T(n) = 64T\left(\frac{n}{8}\right) - n^2 \log_2 n$$

$$a = 64, b = 8, c = 2, p = 1$$

$$\frac{a}{b^c} = \frac{64}{8^2} = 1, \text{ and } p > -1 \text{ then}$$

$$\begin{aligned} T(n) &= \Theta\left(n^{\log_b a} \log_b^{p+1} n\right) \\ &= \Theta\left(n^{\log_8 64} \log_8^2 n\right) \\ &= \Theta(n^2 \log_8^2 n) \end{aligned}$$